

Asie – 2025 – sujet1

Exercice 1 (6 points)

Cet exercice porte sur la décidabilité, l'algorithmique et la programmation en Python.

En Python, on peut utiliser le triple guillemet pour écrire une chaîne de caractères sur plusieurs lignes. Par exemple, on peut définir une variable `programme1` qui contient la chaîne de caractères correspondant à un petit programme Python de la manière suivante :

```
1     programme1 = """
2     x = 10
3     y = 10
4     while x > 0:
5         x = x - 1
6         y = y + 1
7     """
```

De même, on peut définir la variable `programme2` :

```
1     programme2 = """
2     def boucle_infinie():
3         while True:
4             pass # Ne rien faire
5     boucle_infinie()
6     """
```

1. On suppose que l'on exécute le programme contenu dans la variable `programme1`. Donner les valeurs de `x` et de `y` après exécution de ce programme.
2. Expliquer pourquoi tout programme Python peut être vu comme une chaîne de caractères.

En Python, la fonction `exec` permet d'exécuter le programme correspondant à une chaîne de caractères passée en paramètre.

```
>>> exec("r = 42")
>>> r
42
>>> exec(programme1)
>>> x + y
20
>>> exec(programme1)
[ne termine pas]
```

On considère les quatre variables `programme3`, `programme4`, `programme5`, `programme6` suivants :

```
1 programme3 = """
2 x = 10
3 while x != 0:
4     x = x - 2
5 """
6
7 programme4 = """
8 x = 10
9 while x > 0:
10     x = x + 2
11 """
12
13 programme5 = """
14 x = 10
15 while x < 0:
16     x = x + 4
17 """
18
19 programme6 = """
20 x = 10
21 while x != 0:
22     x = x - 4
23 """
```

3. On exécute les variables `programme3`, `programme4`, `programme5`, `programme6` avec la fonction `exec`. Déterminer lesquelles terminent et lesquelles ne terminent pas.

On cherche à écrire une fonction `arret` telle que `arret (programme)` renvoie `True` si `exec (programme)` termine et `False` si `exec (programme)` ne termine pas. Cette fonction `arret` doit donc s'arrêter dans tous les cas.

4. Indiquer ce que réalise le programme suivant et s'il permet de répondre au problème posé ci-dessus.

```
1     def arret_essail (programme) :
2         exec (programme)
3         return True
```

On suppose disposer d'une fonction `recherche (mot, texte)` qui renvoie `True` si une chaîne de caractères `mot` est présente dans une chaîne de caractères `texte` et `False` sinon.

5. Expliquer succinctement le principe de l'algorithme de Boyer-Moore qui permet d'implémenter cette fonction `recherche`.

Une idée est d'écrire une fonction qui décrète qu'un programme s'arrête s'il ne contient pas de `while` et ne s'arrête pas s'il en contient un.

6. Écrire une fonction `arret_essai2 (programme)` qui renvoie `True` si la chaîne de caractères `"while"` n'est pas utilisée dans la chaîne de caractères `programme` et `False` sinon.
7. Montrer qu'il est possible que :
 - `arret_essai2 (programme)` renvoie `True` alors que le programme ne s'arrête pas;
 - `arret_essai2 (programme)` renvoie `False` alors que le programme s'arrête.

Indication : il n'y a pas que les boucles `while` qui peuvent poser des problèmes de non terminaison.

Nos tentatives pour écrire une telle fonction `arret` sont restées vaines. Nous allons montrer qu'il est en réalité *impossible* d'écrire une telle fonction. On va supposer qu'une telle fonction `arret` existe. On va montrer que cette supposition aboutit à un paradoxe ce qui prouvera que la supposition est fausse.

8. Écrire une fonction `terminaison_inverse` telle que l'appel `terminaison_inverse (programme)` termine si la chaîne de caractères `programme` représente un programme qui ne termine pas et ne termine pas si la chaîne de caractères `programme` représente un programme qui termine. On pourra utiliser la fonction `boucle_infinie` de `programme2` ainsi bien sûr que la fonction `arret` dont on a supposé l'existence.

On considère `"terminaison_inverse (programme_paradoxal) "` qui n'est rien d'autre qu'une chaîne de caractères, et on définit une variable que l'on appelle `programme_paradoxal` à laquelle on affecte cette chaîne de caractères : 1
`programme_paradoxal =`
`"terminaison_inverse (programme_paradoxal) "`

9. Étudier si le programme `paradoxal` termine ou non, c'est-à-dire si `exec (programme_paradoxal)` termine ou non.
10. Indiquer ce que l'on peut conclure sur la fonction `arret`.
11. Expliquer si l'impossibilité d'écrire une telle fonction `arret` est due aux limitations du langage Python.